ProcureFlow

Enterprise Procure-to-Pay Platform

Project Plan, Architecture & Technical Documentation

Java 21 | Spring Boot 3 | PostgreSQL | React + TypeScript

Document version	1.0 (initial plan)
Date	25 October 2026
Author	Preet
Status	Draft for implementation
Frontend counterpart	VendorBridge (React)

Table of Contents

1. Executive Summary 4

2. Scope, Actors and Roles 4

2.1 In scope (MVP) . . . 4

2.2 Out of scope (v1) . . . 4

2.3 Actors and roles . . . 4

3. Functional Requirements 5

4. Non-Functional Requirements 5

5. Architecture 6

5.1 Architectural style . . . 6

5.2 Dependency rules (enforced by ArchUnit) . . . 6

5.3 Modules . . . 7

5.4 Package structure of one module (example: purchasing) . . . 8

6. Domain Model and Database 8

6.1 Key aggregates and value objects . . . 8

6.2 Persistence principles . . . 9

6.3 Table catalogue . . . 9

7. Core Workflows 10

7.1 Purchase order state machine . . . 10

7.2 Approval chain . . . 10

7.3 Three-way matching algorithm . . . 11

7.4 Payment flow (idempotent) . . . 11

8. Design Principles and Patterns Map 11

8.1 SOLID . . . 11

8.2 Design patterns . . . 11

8.3 Other principles to state explicitly . . . 12

9. REST API Design 12

9.1 Conventions . . . 12

9.2 Endpoint catalogue (v1) . . . 12

9.3 Sample error response . . . 13

10. Security Design 13

11. Concurrency, Transactions and Events 14

12. Technology Stack 14

13. Repository Layout 14

14. Frontend Plan (React) 15

15. Testing Strategy 15

16. Observability and DevOps **16**

17. Delivery Roadmap (8 Weeks) **16**

Milestones 16

Suggested first tasks (day 1 to 3) 16

18. Risks and Mitigations **17**

19. Definition of Done and Quality Checklist **17**

20. Interview Positioning **17**

20.1 Resume bullets (adapt with your real numbers) 17

20.2 Questions you should be able to answer 17

Appendix A - Code Skeletons **19**

A.1 Strategy: approval rules (Open/Closed) 19

A.2 Aggregate with guarded state transitions and events 19

A.3 Application service (use case) with transaction + outbox 19

A.4 Transactional outbox relay 20

A.5 Idempotent payment endpoint (sketch) 20

A.6 ArchUnit rules 20

Appendix B - Glossary **20**

Appendix C - Recommended Reading **20**

1. Executive Summary

ProcureFlow is a procure-to-pay (P2P) platform that digitises how a company buys things: an employee raises a purchase requisition, it is approved, vendors are invited to quote, a purchase order is issued, goods are received, the vendor invoice is matched against the order and receipt (three-way match), and finally the payment is recorded. Every step is audited, permission-controlled and traceable.

The project is intentionally **backend-heavy**. The React frontend (an extension of the existing VendorBridge UI) is a client of a well-structured Spring Boot system whose purpose is to demonstrate production-grade engineering: clean architecture, DDD, SOLID, design patterns, security, concurrency control, event-driven integration, automated testing and observability.

Goal	How the project demonstrates it
Enterprise-grade backend	Modular monolith with strict module boundaries, hexagonal layering, 12 bounded modules
Design principles	SOLID applied to real problems (approval rules, state transitions, payment methods, notifications)
Production concerns	JWT + RBAC, audit trail, idempotency, optimistic locking, Flyway, Redis cache, rate limiting, outbox
Quality engineering	Unit + integration tests (Testcontainers), architecture tests (ArchUnit), CI pipeline, coverage gates
Operability	Dockerised, health checks, metrics (Micrometer/Prometheus/Grafana), structured logs, correlation ids
Interview value	Rich story: workflows, money, concurrency, events; scales from monolith to services

Guiding rule: build a well-designed modular monolith first. Extract Notification and Billing into separate services (with Kafka) only in the final phase. This is realistic, finishable, and easier to defend in an interview than starting with microservices on day one.

2. Scope, Actors and Roles

2.1 In scope (MVP)

- Vendor onboarding, approval and lifecycle management
- Item catalog, cost centres / departments and budgets
- Purchase requisitions with multi-level, rule-driven approvals
- RFQ creation, vendor quotations, quote comparison and award
- Purchase orders (state machine), revisions, cancellation
- Goods receipt (full and partial) and basic stock movement
- Vendor invoices, three-way matching, discrepancy handling
- Payments (recorded, idempotent), payment status tracking
- Notifications (in-app + email), full audit trail, dashboards and reports (CSV/PDF export)

2.2 Out of scope (v1)

- Real bank integration (payments are simulated through a gateway interface)
- Multi-currency FX engine (single base currency with a currency field kept for extension)
- Full accounting ledger / ERP general ledger posting
- Mobile apps

2.3 Actors and roles

Role	Responsibilities	Typical permissions
Requester (Employee)	Raises requisitions, tracks status	REQ_CREATE, REQ_READ_OWN
Department Manager	First-level approver, department budget view	REQ_APPROVE_L1, BUDGET_READ

Role	Responsibilities	Typical permissions
Finance Approver	Approves high-value spend, invoices, payments	REQ_APPROVE_L2, INVOICE_APPROVE, PAYMENT_CREATE
Procurement Officer	Runs RFQs, creates POs, manages vendors	RFQ_*, PO_*, VENDOR_*
Warehouse Clerk	Records goods receipts	GRN_CREATE, STOCK_READ
Vendor (external)	Views POs, submits quotations and invoices	VENDOR_PORTAL_*
Admin	Users, roles, rules, system configuration	ADMIN_*
Auditor (read-only)	Reads everything including audit log	AUDIT_READ, *_READ

3. Functional Requirements

Requirement ids are used in tests and in the roadmap so every feature is traceable. Priority: **M** = must, **S** = should, **C** = could.

ID	Requirement	Pri
FR-IAM-1	Users authenticate with email + password; receive short-lived access token and rotating refresh token	M
FR-IAM-2	Roles map to fine-grained permissions; permissions enforced at method level	M
FR-IAM-3	Account lockout after repeated failures; password policy; optional TOTP 2FA	S
FR-VEN-1	Create vendor (PENDING), review, approve/reject/suspend; vendor documents upload	M
FR-VEN-2	Tax id unique; bank details stored encrypted; changes to bank details require re-approval	M
FR-CAT-1	Maintain catalog items, unit of measure, category, preferred vendors	M
FR-BUD-1	Department budgets per fiscal period; commit on approval, release on cancel, consume on invoice	M
FR-REQ-1	Requester creates requisition with lines, attachments and justification; save as draft	M
FR-REQ-2	Submit triggers approval chain chosen by rules (amount, category, department)	M
FR-APR-1	Approvers approve/reject with comment; delegation and escalation after SLA timeout	M
FR-APR-2	Approval rules are configurable data + pluggable strategies (no code change for thresholds)	S
FR-RFQ-1	Create RFQ from requisition, invite vendors, set deadline; vendors submit quotations	M
FR-RFQ-2	Comparison matrix (price, lead time, rating); award one or split-award	S
FR-PO-1	Create PO from awarded quotation or directly; sequential, gap-free PO numbers	M
FR-PO-2	PO follows state machine; invalid transitions rejected with domain error	M
FR-PO-3	PO revision creates new version; history retained	S
FR-GRN-1	Record full/partial receipts; cannot over-receive beyond tolerance	M
FR-INV-1	Register vendor invoice; unique per (vendor, invoice number)	M
FR-INV-2	Three-way match PO vs GRN vs invoice with configurable price/qty tolerance	M
FR-PAY-1	Pay approved invoices; supports partial payments; idempotent via Idempotency-Key	M
FR-NOT-1	Event-driven in-app and email notifications; user preferences	S
FR-AUD-1	Immutable audit log: who, what, when, before/after, ip, correlation id	M
FR-RPT-1	Dashboards: spend by vendor/category, cycle time, pending approvals, budget utilisation	S
FR-RPT-2	Export reports to CSV/PDF with background generation for large sets	C

4. Non-Functional Requirements

Area	Target / approach
Performance	p95 < 300 ms for read endpoints at 200 concurrent users; list APIs paginated and indexed; N+1 queries prevented (checked in tests)

Area	Target / approach
Scalability	Stateless app nodes (horizontal scaling); Redis for shared cache; DB read replica ready; async work via events
Reliability	Transactional outbox for no-lost events; idempotent consumers; retries with backoff; graceful shutdown
Consistency	ACID inside an aggregate; eventual consistency across modules via events; optimistic locking on aggregates
Security	OWASP ASVS L2 mindset: JWT, RBAC, input validation, parameterised queries, rate limits, secrets in env/vault, encrypted sensitive fields
Auditability	Every state change recorded; audit log append-only; entity revisions via Hibernate Envers
Maintainability	Layered/hexagonal architecture; ArchUnit rules; 80%+ line coverage on domain and application layers
Observability	Structured JSON logs, correlation id, Prometheus metrics, health/readiness probes, optional OpenTelemetry tracing
Portability	Docker Compose for local; same image deployable to EC2/ECS/Kubernetes; 12-factor configuration
Usability / API	Consistent REST conventions, RFC 7807 problem details, OpenAPI documentation, versioned under /api/v1

5. Architecture

5.1 Architectural style

ProcureFlow is a **modular monolith** using **hexagonal (ports and adapters) architecture** inside each module and **Domain-Driven Design** for modelling. One deployable unit keeps operations simple, while strict module boundaries make later extraction into services a mechanical step rather than a rewrite.

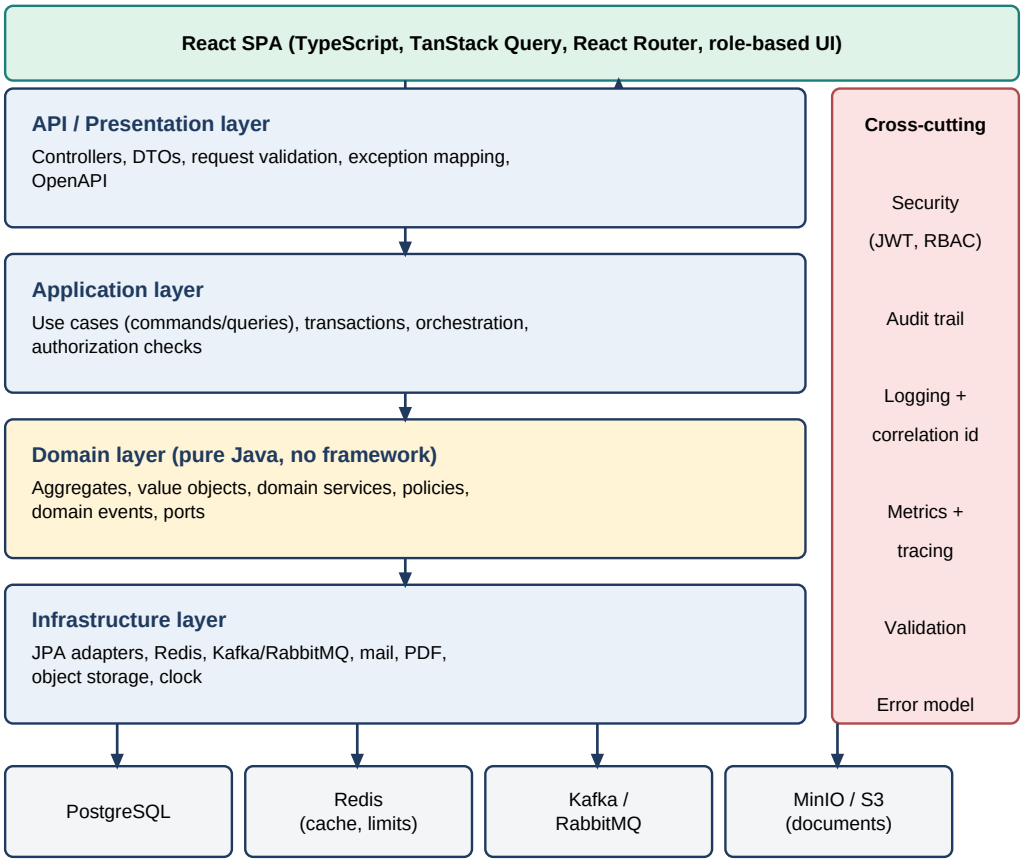


Figure 1 - Layered view. Dependencies point inward: Infrastructure and API depend on Application, Application depends on Domain.

5.2 Dependency rules (enforced by ArchUnit)

- The **domain** package must not import Spring, JPA, Jackson or any infrastructure type.

- **Controllers** only call application services (use cases); they never touch repositories or entities directly.
- A module may access another module only through its **public API package** (interfaces + DTOs) or by listening to its domain events.
- No module reads another module's tables. Cross-module reads use the API or a local read model.
- Cyclic dependencies between modules are forbidden.

5.3 Modules

Blue = core business modules | Amber = supporting modules (reacts to events)

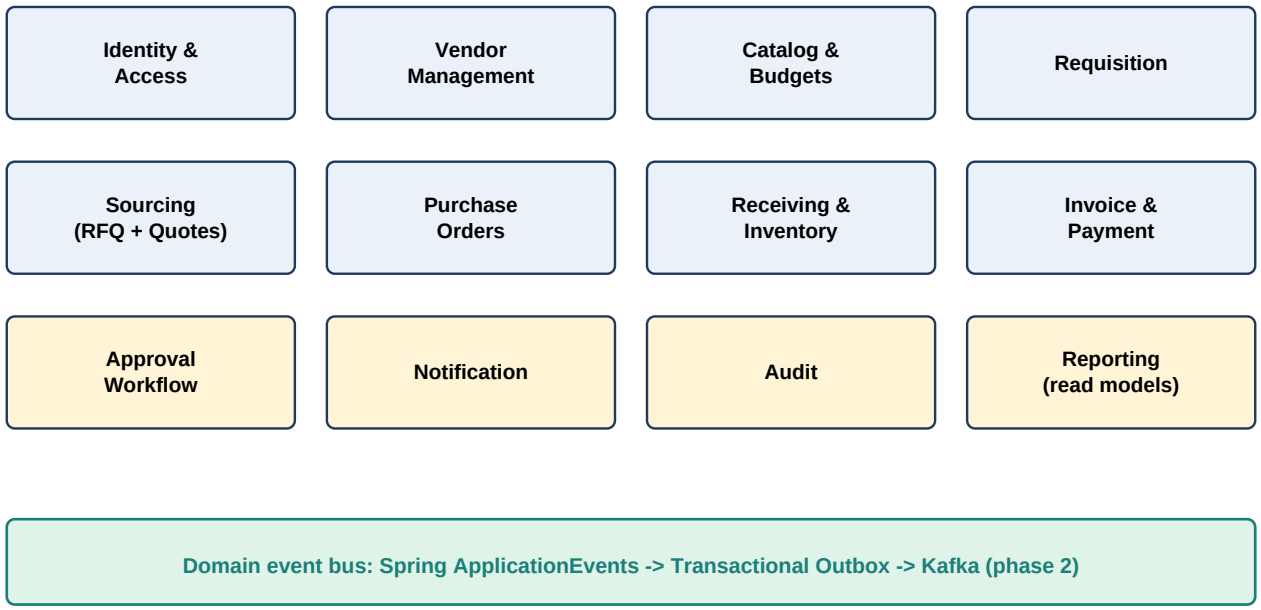


Figure 2 - Module map. Supporting modules react to events and never block the core flow.

Module	Owns (aggregates)	Publishes events	Consumes events
identity	User, Role, Permission, RefreshToken	UserRegistered, UserLocked	-
vendor	Vendor, VendorDocument, VendorContact	VendorApproved, VendorSuspended	-
catalog	Item, Category, Budget	BudgetExceeded	RequisitionApproved, InvoiceMatched
requisition	PurchaseRequisition	RequisitionSubmitted, RequisitionApproved	ApprovalCompleted
approval	ApprovalInstance, ApprovalStep, Delegation	ApprovalCompleted, ApprovalEscalated	RequisitionSubmitted, InvoiceMismatch
sourcing	Rfq, Quotation	RfqPublished, QuotationAwarded	RequisitionApproved
purchasing	PurchaseOrder (+lines, versions)	PurchaseOrderIssued, PurchaseOrderCancelled	QuotationAwarded
receiving	GoodsReceipt, StockMovement	GoodsReceived	PurchaseOrderIssued
billing	Invoice, MatchResult, Payment	InvoiceMatched, InvoiceMismatch, PaymentCompleted	GoodsReceived
notification	NotificationTemplate, Notification, Preference	-	almost all events
audit	AuditEntry	-	all events + AOP hooks
reporting	Read models / materialised views	-	all business events

5.4 Package structure of one module (example: purchasing)

```
com.procureflow.purchasing
 |-- api                      <- public contract other modules may use
 |    |-- PurchaseOrderApi.java        (interface)
 |    |-- PurchaseOrderSummary.java    (DTO)
 |    `-- events/PurchaseOrderIssued.java
 |-- domain                   <- pure Java, no framework imports
 |    |-- PurchaseOrder.java           (aggregate root)
 |    |-- PoLine.java, PoNumber.java, Money.java   (entity / value objects)
 |    |-- PoStatus.java                (state machine)
 |    |-- service/PoPricingPolicy.java (domain service)
 |    `-- port/PurchaseOrderRepository.java   (outbound port)
 |-- application              <- use cases
 |    |-- command/IssuePurchaseOrderHandler.java
 |    |-- query/PurchaseOrderQueryService.java
 |    `-- dto/...
 |-- infrastructure           <- adapters
 |    |-- persistence/JpaPurchaseOrderRepository.java, PoEntity.java, PoMapper.java
 |    `-- messaging/PoEventPublisher.java
 `-- web                      <- inbound adapter
      |-- PurchaseOrderController.java
      `-- request/IssuePoRequest.java, response/PurchaseOrderResponse.java
```

6. Domain Model and Database

6.1 Key aggregates and value objects

- **Aggregates:** PurchaseRequisition, Rfq, PurchaseOrder, GoodsReceipt, Invoice, Payment, ApprovalInstance, Vendor, Budget.
- **Value objects (immutable):** Money (amount + currency, BigDecimal arithmetic), Quantity (value + unit), PoNumber, TaxId, EmailAddress, DateRange.
- **Domain services:** ThreeWayMatchService, BudgetCheckService, PoNumberGenerator, VendorScoringService.
- **Rules live in the domain:** an aggregate method such as *purchaseOrder.approve(by)* checks invariants and raises events; the service layer only orchestrates.

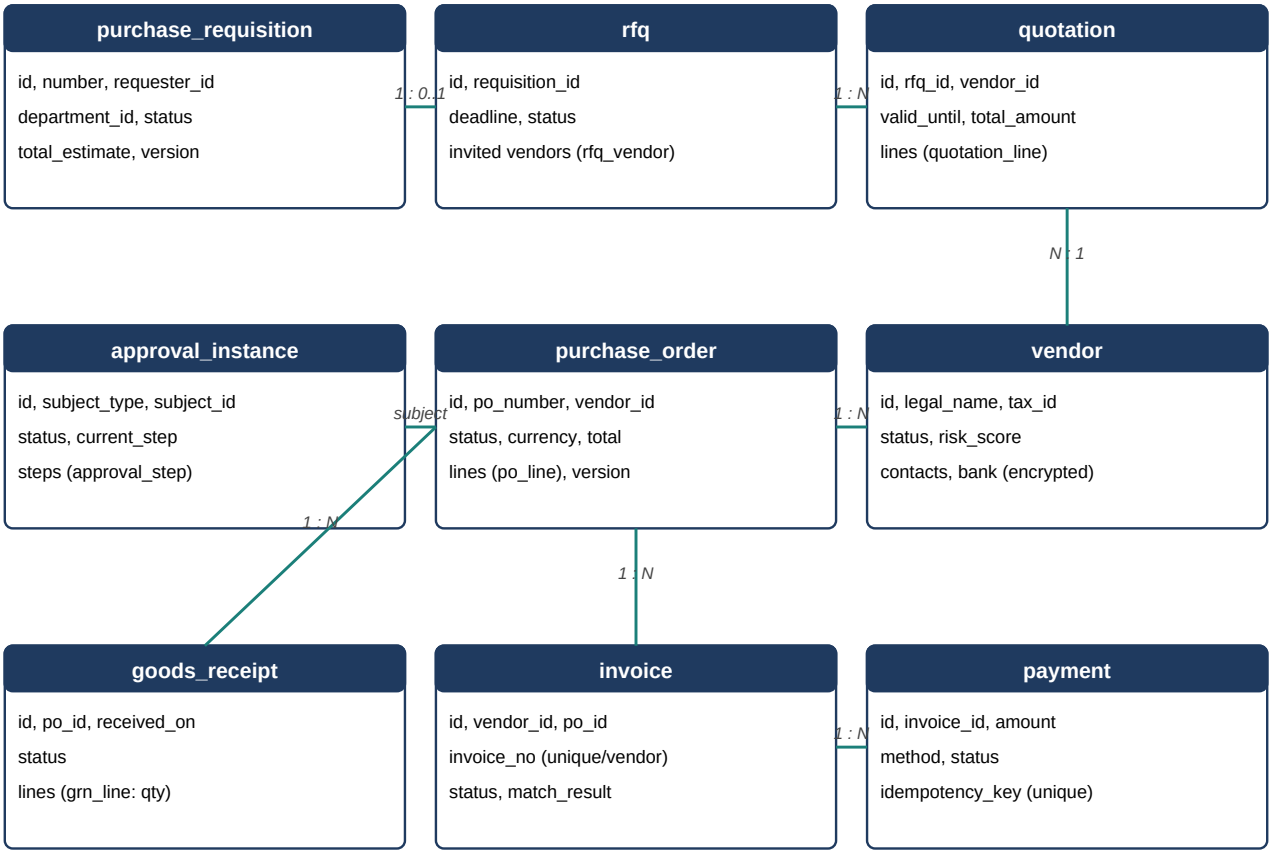


Figure 3 - Core entity relationship overview (simplified; audit and outbox tables omitted).

6.2 Persistence principles

- **PostgreSQL 16** with **Flyway** migrations (V1__init.sql, V2__...). Hibernate *ddl-auto=validate* only.
- Money stored as NUMERIC(19,4); never float/double. Currency kept in a separate column.
- Every aggregate root has **@Version** for optimistic locking; conflicting updates return HTTP 409.
- Soft delete (deleted_at) for master data; documents are never physically deleted.
- Auditing columns: created_at, created_by, updated_at, updated_by through Spring Data auditing. Hibernate Envers keeps revision history for PO and Invoice.
- Sequential PO numbers from a dedicated DB sequence per fiscal year (gap-free requirement handled via a numbering table with row lock).
- Indexes designed per query: (vendor_id, status), (status, created_at), partial unique index on invoice(vendor_id, invoice_no).
- Large reports read from **read-model tables/materialised views** refreshed by event handlers, not by joining the transactional model.

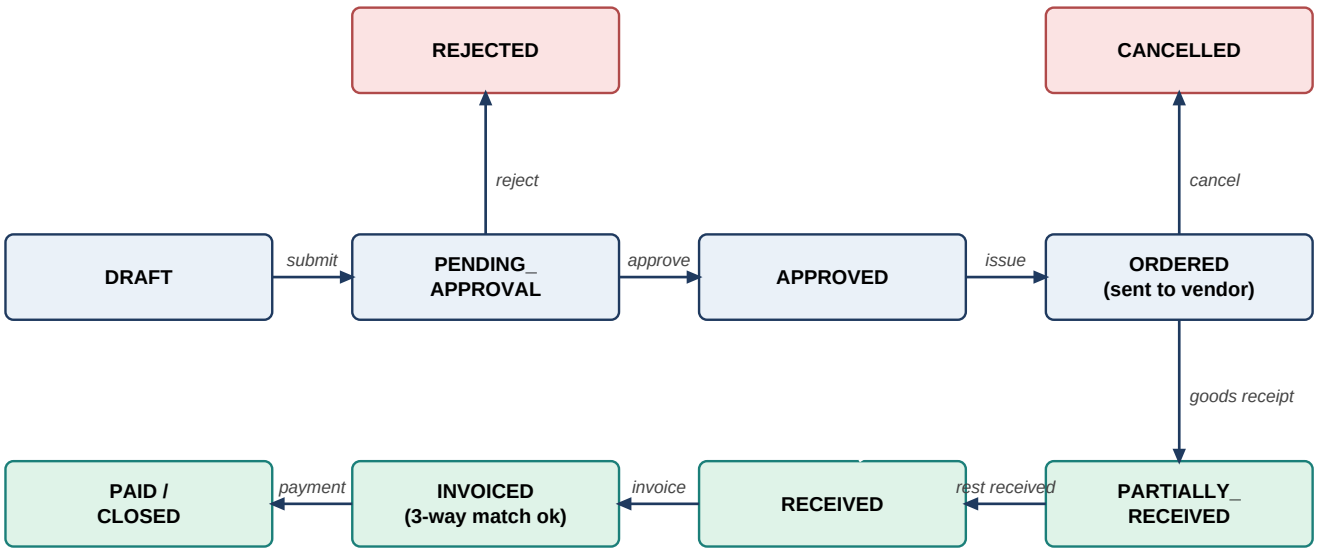
6.3 Table catalogue

Table	Important columns / constraints
app_user, role, permission, user_role, role_permission	email unique, password_hash (BCrypt/Argon2), status, failed_attempts, locked_until
refresh_token	token_hash, user_id, expires_at, revoked, replaced_by (rotation chain)
vendor, vendor_contact, vendor_document	tax_id unique, status, bank_account_enc, rating
item, category, department, budget	budget(department_id, fiscal_year, allocated, committed, consumed) with check committed+consumed <= allocated

Table	Important columns / constraints
purchase_requisition, requisition_line	status, requester_id, department_id, estimate_total, version
approval_instance, approval_step, delegation	subject_type + subject_id, step_order, approver_id, decision, decided_at, sla_due_at
rfq, rfq_vendor, quotation, quotation_line	deadline, status, awarded_quotation_id, valid_until
purchase_order, po_line, po_version	po_number unique, vendor_id, status, total, version; po_line(qty_ordered, qty_received, unit_price)
goods_receipt, grn_line	po_id, received_by, received_on, qty_received per po_line
invoice, invoice_line, match_result	unique(vendor_id, invoice_no), status, match_status, variance_amount
payment	invoice_id, amount, status, method, idempotency_key unique, gateway_ref
audit_entry	actor, action, entity_type, entity_id, before_json, after_json, ip, correlation_id (append-only)
outbox_event	aggregate_type, aggregate_id, event_type, payload_json, created_at, published_at, attempts
processed_event	consumer, event_id (primary key pair) for idempotent consumers
notification, notification_preference	user_id, channel, template_code, status, sent_at

7. Core Workflows

7.1 Purchase order state machine



REJECTED -> DRAFT (requester edits and resubmits). Receipts can arrive in several partial batches.

Figure 4 - Purchase order lifecycle. The PurchaseOrder aggregate is the only code allowed to change its status.

Transitions are encoded in the aggregate (State pattern / guarded methods). Each successful transition appends an audit entry and an outbox event in the same database transaction.

7.2 Approval chain

- On *RequisitionSubmitted* the ApprovalPolicyService collects every ApprovalRule bean that applies and builds an ordered list of steps (e.g. Manager, then Finance if total > 100,000, then Director if total > 500,000).
- A step may be satisfied by a role (any Finance Approver) or a specific user; an absent approver can delegate for a date range.

- Each step has an SLA; a scheduled job escalates overdue steps to the next-level approver and emits *ApprovalEscalated*.
- Self-approval is blocked: requester can never approve their own requisition (domain invariant).
- When the last step approves, the instance emits *ApprovalCompleted* and the requisition moves to APPROVED; budget is committed in the same transaction.

7.3 Three-way matching algorithm

Triggered when an invoice is registered or when a new goods receipt arrives for an invoiced PO.

- For each invoice line find the PO line. Missing line leads to MISMATCH (reason UNORDERED_ITEM).
- **Quantity check:** invoiced qty must be <= received qty (accepted GRN) minus already-invoiced qty.
- **Price check:** |invoice unit price - PO unit price| / PO unit price <= price tolerance (default 2%).
- **Total check:** header total equals sum of lines plus tax within rounding tolerance.
- Result is MATCHED, MATCHED_WITH_TOLERANCE or MISMATCH. MISMATCH creates a task for Procurement and notifies the vendor; MATCHED moves the invoice to APPROVED_FOR_PAYMENT.
- Tolerances are configuration data (per vendor/category) injected into the ThreeWayMatchService as a policy object.

7.4 Payment flow (idempotent)

- Client sends *POST /api/v1/payments* with header *Idempotency-Key*.
- Server stores key + request hash in a unique-constrained table; a repeat with the same key returns the stored response, a repeat with a different body returns 422.
- PaymentGateway is a port; v1 adapter is a simulator with configurable failures to test retry and compensation paths.
- Status: INITIATED -> PROCESSING -> COMPLETED | FAILED. A failed payment can be retried; the invoice becomes PAID only when total completed payments equal the invoice amount.

8. Design Principles and Patterns Map

This is the heart of the project: each principle has a concrete place in the code, so it can be shown, tested and explained.

8.1 SOLID

Principle	Where it is applied	Concrete example
S - Single Responsibility	One use case per handler class; controllers only translate HTTP	IssuePurchaseOrderHandler issues a PO; it does not send mail or write audit rows (listeners do)
O - Open/Closed	Approval rules, match tolerance rules, payment methods, notification channels	Add a new ApprovalRule bean and it is picked up; no edit to ApprovalPolicyService
L - Liskov Substitution	Payment gateways, storage providers, notification senders behave per their contract	StripeLikeGateway and SimulatorGateway are interchangeable; contract tests run against both
I - Interface Segregation	Small ports instead of fat repositories	PoReader (queries) and PoWriter (commands) separated; reporting depends only on readers
D - Dependency Inversion	Domain defines ports; infrastructure implements	PurchaseOrderRepository interface in domain; JPA adapter in infrastructure; Clock and IdGenerator injected

8.2 Design patterns

Pattern	Use in ProcureFlow
Strategy	ApprovalRule, PricingPolicy, MatchTolerancePolicy, PaymentMethodStrategy, ReportExporter (CSV/PDF)

Pattern	Use in ProcureFlow
State	PurchaseOrder, Invoice and Payment lifecycles with guarded transitions
Chain of Responsibility	Approval steps; request filters (correlation id, auth, rate limit, idempotency)
Factory / Abstract Factory	DocumentFactory creating PO/Invoice numbers; NotificationSenderFactory per channel
Builder	Complex aggregates and test data builders (PurchaseOrderBuilder)
Observer / Domain Events	Spring ApplicationEventPublisher, then Outbox, then Kafka
Template Method	AbstractCsvImporter for vendor/item bulk import; report generation skeleton
Adapter / Ports and Adapters	Email, PDF, storage, payment gateway, message broker
Decorator	CachingVendorLookup wraps VendorLookup; AuditingPaymentGateway wraps PaymentGateway
Specification	Dynamic search filters (JPA Specifications / Querydsl) for list endpoints
Repository + Unit of Work	Spring Data repositories behind domain ports; @Transactional boundary at use-case level
Command / CQRS-lite	Command handlers mutate; query services return read-optimised DTOs
Outbox + Idempotent Consumer	Reliable event publishing and safe re-delivery
Saga (phase 2)	Cross-service flow PO issued -> reserve budget -> notify vendor with compensation

8.3 Other principles to state explicitly

- **DRY / KISS / YAGNI** - no framework creation; abstractions only when two real use cases exist.
- **Law of Demeter, Tell-Don't-Ask** - aggregates expose behaviour, not setters. No anemic domain model.
- **Fail fast** - constructors and factory methods validate invariants; value objects are always valid.
- **Composition over inheritance** - inheritance only for sealed domain hierarchies (e.g. DiscrepancyReason).
- **Immutability** - Java records for DTOs, events and value objects.
- **Principle of least privilege** - permissions per action; DB user with minimum grants.
- **12-factor app** - config from environment, logs to stdout, stateless processes.

9. REST API Design

9.1 Conventions

- Base path **/api/v1**; plural nouns; actions that are not CRUD use sub-resources, e.g. *POST /purchase-orders/{id}/approve*.
- Pagination: *?page=0&size=20&sort=createdAt,desc* with a stable response envelope (content, page, size, totalElements).
- Filtering through query params mapped to Specifications; whitelisted sort fields only.
- Errors follow **RFC 7807 problem+json** with a stable *code* field (e.g. PO_INVALID_TRANSITION).
- HTTP semantics: 201 + Location on create, 204 on delete, 409 on version conflict or state conflict, 422 on business rule violation, 429 on rate limit.
- Concurrency: ETag / If-Match for updates (maps to @Version). Idempotency-Key for payments and POST creates that clients may retry.
- Every response carries *X-Correlation-Id*; the same id appears in all logs of that request.

9.2 Endpoint catalogue (v1)

Method + path	Purpose	Permission
POST /auth/login, /auth/refresh, /auth/logout	Authentication and token rotation	public / auth
GET/POST /users, PATCH /users/{id}/roles	User and role administration	ADMIN_*
GET/POST /vendors, POST /vendors/{id}/approve | suspend	Vendor lifecycle	VENDOR_*
GET/POST /items, /categories, /departments, /budgets	Master data	CATALOG_*

Method + path	Purpose	Permission
POST /requisitions, PUT /requisitions/{id}, POST /requisitions/{id}/submit	Create, edit draft, submit	REQ_CREATE
GET /requisitions?status=&department=	Search with filters	REQ_READ
GET /approvals/inbox, POST /approvals/{id}/decision	Approver task list and decision	REQ_APPROVE_*
POST /rfqs, POST /rfqs/{id}/publish, POST /rfqs/{id}/award	Sourcing	RFQ_*
POST /rfqs/{id}/quotations (vendor portal)	Submit quotation	VENDOR_PORTAL
POST /purchase-orders, POST /{id}/issue | cancel | revise	PO commands	PO_*
GET /purchase-orders/{id}/timeline	Audit-based history of a PO	PO_READ
POST /purchase-orders/{id}/receipts	Create goods receipt	GRN_CREATE
POST /invoices, GET /invoices/{id}/match-result, POST /{id}/approve	Invoice handling	INVOICE_*
POST /payments (Idempotency-Key), GET /payments/{id}	Payments	PAYMENT_*
GET /reports/spend-by-vendor | cycle-time | budget-utilisation	Analytics	REPORT_READ
GET /audit?entityType=&entityId=	Audit trail search	AUDIT_READ
GET /actuator/health | prometheus	Operations (restricted)	internal

9.3 Sample error response

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json
{
  "type": "https://procureflow.dev/errors/po-invalid-transition",
  "title": "Invalid purchase order transition",
  "status": 422,
  "code": "PO_INVALID_TRANSITION",
  "detail": "Cannot issue purchase order PO-2026-000123 in status DRAFT",
  "correlationId": "7f0d6c1e-3b1a-4f0a-8a9b-1c2d3e4f5a6b",
  "timestamp": "2026-10-25T09:41:12Z"
}
```

10. Security Design

Concern	Design
Authentication	Spring Security, stateless JWT access token (10-15 min) + rotating refresh token stored hashed; refresh reuse detection revokes the token family
Password handling	BCrypt or Argon2id with strong cost factor; breached/weak password check; lockout and exponential delay on failed attempts
Authorization	RBAC with permissions as authorities; *@PreAuthorize* on use cases; ownership checks (requester sees own); separation of duties (creator cannot approve)
Data protection	TLS everywhere; vendor bank details encrypted at rest (AES-GCM, key from environment/KMS); sensitive fields masked in logs
Input validation	Bean Validation on DTOs; whitelisting of sort/filter fields; file upload checks (size, type, antivirus hook); parameterised queries only
Web protections	CORS allow-list, security headers, CSRF disabled only because tokens are not cookies (or enabled with SameSite cookies if used), request size limits
Abuse control	Bucket4j + Redis rate limiting on login and write endpoints; per-IP and per-user limits
Secrets	Environment variables / Docker secrets; none in Git; *.env.example* documents required keys
Audit and forensics	Security events (login, failure, role change, permission denied) written to audit log with ip and user agent
Dependency security	OWASP Dependency-Check or Dependabot and image scanning (Trivy) in CI

11. Concurrency, Transactions and Events

- **Transaction boundary** = one use case = one aggregate change. Cross-aggregate work is done through events (eventual consistency).
- **Optimistic locking** on aggregates; retry policy only for safe idempotent commands.
- **Pessimistic lock** (SELECT ... FOR UPDATE) reserved for numbering tables and budget commit where contention is real.
- **Budget commit** uses an atomic conditional update (UPDATE ... WHERE allocated - committed - consumed >= :amount) to avoid overspend under concurrency.
- **Transactional outbox:** events are inserted in the same transaction as the state change; a poller/relay publishes them and marks published_at. This guarantees at-least-once delivery.
- **Idempotent consumers:** processed_event table prevents double handling after redelivery.
- **Scheduling:** ShedLock (or DB lock) so SLA-escalation jobs run on one node only.
- **Async work:** report generation and email sending run on bounded thread pools (virtual threads in Java 21 are an option) with timeouts.

Test it, do not just claim it: include a concurrency integration test that fires 50 parallel budget commits and asserts that the committed amount never exceeds the allocation, and a test that re-delivers the same event twice.

12. Technology Stack

Layer	Choice	Notes
Language / runtime	Java 21 LTS	Records, sealed types, pattern matching, virtual threads
Framework	Spring Boot 3.x	Web, Validation, Security, Data JPA, Actuator, Cache; check the latest stable 3.x/4.x when you start
Database	PostgreSQL 16 + Flyway	JSONB for audit snapshots; partial indexes
Cache / limits	Redis	Spring Cache, Bucket4j, idempotency, token deny-list
Messaging	Spring Events -> Kafka or RabbitMQ	Kafka in phase 2 (docker-compose)
Mapping	MapStruct, Lombok (sparingly)	Records preferred for DTOs
Auth	Spring Security + jjwt / Nimbus	OAuth2 resource server style config
Docs	springdoc-openapi	Swagger UI, generated TypeScript client for React
Testing	JUnit 5, Mockito, AssertJ, Testcontainers, ArchUnit, RestAssured/MockMvc, JaCoCo, Pitest (optional)	Test pyramid below
Observability	Micrometer, Prometheus, Grafana, Loki or ELK, OpenTelemetry (optional)	Dashboards committed as JSON
Build / CI	Maven (multi-module or single module with packages), GitHub Actions, Docker, SonarQube (optional)	Reuse your CI/CD pipeline project
Frontend	React 18, TypeScript, Vite, React Router, TanStack Query, React Hook Form + Zod, your VendorBridge design system	Generated API client
Infrastructure	Docker Compose locally; AWS EC2/ECS + RDS + S3 + Nginx for deployment	Terraform optional

13. Repository Layout

```
procureflow/
 |-- backend/
 |    |-- pom.xml
 |    |-- src/main/java/com/procureflow/
```

```
|    |    |-- ProcureFlowApplication.java
|    |    |-- shared/            (kernel: Money, Result, ids, base events, error model)
|    |    |-- config/            (security, jackson, cache, async, openapi)
|    |    |-- identity/ vendor/ catalog/ requisition/ approval/ sourcing/
|    |    |-- purchasing/ receiving/ billing/ notification/ audit/ reporting/
|    |-- src/main/resources/
|    |    |-- application.yml, application-{dev,test,prod}.yml
|    |    `-- db/migration/V1__init.sql ...
|    `-- src/test/java/...      (unit, integration, architecture, concurrency)
|-- frontend/                   (React + TypeScript, based on VendorBridge)
|-- docker/                     (Dockerfile, compose.yml, nginx.conf, grafana/, prometheus/)
|-- docs/                       (ADRs, C4 diagrams, API examples, runbook)
|-- .github/workflows/          (ci.yml, release.yml)
`-- README.md
```

14. Frontend Plan (React)

The UI is deliberately thinner than the backend but must prove that the API is usable and that role-based behaviour is correct.

Area	Screens	Highlights
Auth	Login, session expiry, profile	Token refresh interceptor, protected routes, permission-aware navigation
Requester	My requisitions, create/edit wizard, status timeline	Form validation (Zod), draft autosave, attachments
Approver	Approval inbox, decision dialog, delegation	Optimistic updates with rollback, badge counts
Procurement	Vendors, RFQ builder, quote comparison matrix, PO list/detail	Data tables with server-side paging and filters
Warehouse	Receive goods against PO	Partial receipt entry, over-receipt guard
Finance	Invoices, match result view, payments	Discrepancy highlighting, idempotent pay button
Dashboards	Spend, cycle time, budget utilisation	Charts, CSV/PDF export
Admin	Users and roles, rules, audit log viewer	Audit diff viewer (before/after)

- State: TanStack Query for server state; minimal local state; URL as the source of truth for filters.
- Generate the TypeScript API client from OpenAPI so contracts cannot drift.
- Error handling: global handler maps problem+json codes to user messages; 401 triggers refresh; 403 shows an access-denied view.
- Testing: Vitest + React Testing Library for components, Playwright for 3 end-to-end journeys (requisition to payment, rejection path, mismatch path).

15. Testing Strategy

Level	What	Tools	Target
Unit	Aggregates, value objects, policies, strategies (no Spring context)	JUnit 5, AssertJ, Mockito	80%+ on domain/application
Slice	Controllers (MockMvc), JPA repositories	@WebMvcTest, @DataJpaTest	All endpoints
Integration	Full flows with real PostgreSQL/Redis/Kafka	Testcontainers, RestAssured	Critical paths
Architecture	Layering and module rules	ArchUnit	Run in CI, fail build
Concurrency	Parallel budget commit, double payment, duplicate events	ExecutorService, CountDownLatch	Key invariants
Contract	Gateway and sender port contracts	Abstract test classes	All adapters
Performance	Load test list and create endpoints	k6 or Gatling	p95 target (section 4)
Mutation (optional)	Test quality in domain package	Pitest	>= 70% mutation score
End-to-end UI	Three user journeys	Playwright	Run on main

Quality gates in CI: build, unit and integration tests, JaCoCo threshold, ArchUnit, dependency scan, Docker image build and push. Merge to main is blocked if any gate fails.

16. Observability and DevOps

- **Logging:** JSON logs with timestamp, level, correlationId, userId, module; no PII or secrets.
- **Metrics:** JVM and HTTP metrics plus business metrics (pos_issued_total, approvals_pending, invoice_mismatch_total, outbox_lag_seconds).
- **Health:** liveness and readiness probes including DB, Redis and broker checks.
- **Dashboards and alerts:** Grafana dashboards for latency, error rate, outbox lag, and failed payments; alert rules committed as code.
- **Deployment:** multi-stage Dockerfile (distroless or JRE slim), non-root user; Compose for local; GitHub Actions to build, push, deploy to EC2 behind Nginx with TLS; rollbacks by image tag.
- **Configuration:** profiles dev/test/prod; secrets from environment; Flyway runs on startup in dev and as a controlled step in prod.
- **Documentation as code:** ADRs in /docs/adr, C4 diagrams (Structurizr or Mermaid), runbook for common incidents.

17. Delivery Roadmap (8 Weeks)

Assumes roughly 15 to 20 focused hours per week. Each phase ends with a demo-able increment and a tagged release.

Week	Focus	Deliverables	Exit criteria
1	Foundation	Repo, Maven setup, Docker Compose (Postgres, Redis), Flyway baseline, shared kernel (Money, errors), CI skeleton, ArchUnit rules, OpenAPI	App boots in Docker, CI green, first ADRs written
2	Identity and vendors	Users/roles/permissions, JWT + refresh rotation, vendor module with approval, audit foundation (AOP + Envers)	Login flow works, RBAC tests pass, vendor lifecycle tested
3	Catalog, budgets, requisitions	Items, departments, budgets with atomic commit, requisition aggregate and API, React login + requisition screens	Create and submit requisition end to end
4	Approval engine	Rule strategies, step builder, delegation, SLA escalation job, inbox API and UI	Rules configurable via data; self-approval blocked; concurrency test green
5	Sourcing and purchasing	RFQ, quotations, comparison, award; PO aggregate with state machine, PO numbering, revisions	Requisition to issued PO with full audit timeline
6	Receiving and billing	Goods receipt, stock movement, invoice registration, three-way match engine, payment with idempotency, simulator gateway	Happy path and mismatch path demo
7	Events, notifications, reporting	Outbox + relay, idempotent consumers, notification module, read models, dashboards and exports	No lost events under failure test; dashboards live
8	Hardening and release	Load test, security review, Kafka extraction of Notification (optional), Grafana alerts, docs, demo video, README, resume bullets	Tagged v1.0, deployed publicly, docs complete

Milestones

- **M1 (end week 3):** secured API + requisition flow with frontend.
- **M2 (end week 5):** requisition to issued PO with approvals.
- **M3 (end week 6):** complete procure-to-pay happy path.
- **M4 (end week 8):** production-ready release with observability and documentation.

Suggested first tasks (day 1 to 3)

- Create repository, choose package structure, write ADR-001 (modular monolith) and ADR-002 (hexagonal + DDD).
- Docker Compose with PostgreSQL and Redis; Spring Boot skeleton with Flyway V1 migration.
- Implement shared kernel: Money value object with tests, ErrorCode enum, ProblemDetail handler, CorrelationIdFilter.
- Add ArchUnit test that fails if domain imports Spring, then wire the GitHub Actions workflow.
- Implement identity module first; everything else depends on authentication and authorization.

18. Risks and Mitigations

Risk	Impact	Mitigation
Scope creep (too many modules)	Never finishes	Protect the MVP list; mark C-priority items optional; time-box each week
Over-engineering patterns	Hard to read code	Apply a pattern only where a real variation exists; document why in ADRs
Complex approval rules	Delays in weeks 4 to 5	Start with two rules (amount, department), add others after demo
Distributed complexity too early	Operational burden	Stay monolith; Kafka only after outbox is stable
Weak tests on money/concurrency	Hidden bugs	Test invariants first (TDD for Money, budget, matching)
UI consuming time	Backend depth suffers	Reuse VendorBridge components; keep UI to required journeys
Environment issues	Lost days	Everything runs through Docker Compose; one command to start

19. Definition of Done and Quality Checklist

- Feature has acceptance criteria linked to FR ids and automated tests at the right level.
- Domain rules live in the domain layer; no business logic in controllers or entities' setters.
- Flyway migration included; rollback/forward plan noted for risky changes.
- Authorization annotated and tested (positive and negative cases).
- Audit entry and event produced for every state change.
- Errors return problem+json with stable codes; OpenAPI updated.
- Logs and metrics added for new flows; no sensitive data logged.
- CI green including ArchUnit and coverage gates; documentation or ADR updated.

20. Interview Positioning

20.1 Resume bullets (adapt with your real numbers)

- Designed and built ProcureFlow, a procure-to-pay platform (Spring Boot 3, PostgreSQL, Redis, React) as a modular monolith using hexagonal architecture and DDD with 12 bounded modules.
- Implemented rule-driven multi-level approval engine (Strategy + Chain of Responsibility) and a three-way invoice matching engine with configurable tolerances.
- Ensured data integrity under concurrency using optimistic locking, atomic budget commits and idempotent payments; verified by parallel integration tests.
- Built reliable event-driven integration with a transactional outbox and idempotent consumers; added Prometheus/Grafana monitoring and a CI pipeline with ArchUnit and Testcontainers.

20.2 Questions you should be able to answer

- Why a modular monolith instead of microservices? When would you split it?
- How do you prevent two approvers or two payments from racing? What is optimistic vs pessimistic locking here?
- What happens if the broker is down when an order is issued? (Outbox.)
- How does the domain stay independent from Spring/JPA, and what does that cost you?

- Show where Open/Closed pays off in the approval engine.
- How would you scale reads for reporting? (Read models, replicas, caching.)
- How do you handle a bug that corrupts data? (Audit, Envers, replay of events, migrations.)

Appendix A - Code Skeletons

A.1 Strategy: approval rules (Open/Closed)

```
public interface ApprovalRule {
    boolean appliesTo(ApprovalContext ctx);
    List<ApprovalStepSpec> steps(ApprovalContext ctx);
    int order();                        // deterministic ordering of contributed steps
}

@Component
class AmountThresholdRule implements ApprovalRule {
    private final ApprovalThresholdProvider thresholds;   // data driven, cached
    public boolean appliesTo(ApprovalContext c) { return c.total().isPositive(); }
    public List<ApprovalStepSpec> steps(ApprovalContext c) {
        return thresholds.levelsFor(c.department(), c.total()).stream()
                .map(l -> ApprovalStepSpec.forRole(l.role(), l.slaHours()))
                .toList();
    }
    public int order() { return 10; }
}

@Service
class ApprovalPolicyService {
    private final List<ApprovalRule> rules;    // all beans injected, no edits to add rules
    ApprovalPolicyService(List<ApprovalRule> rules) {
        this.rules = rules.stream().sorted(comparingInt(ApprovalRule::order)).toList();
    }
    List<ApprovalStepSpec> build(ApprovalContext ctx) {
        return rules.stream().filter(r -> r.appliesTo(ctx))
                .flatMap(r -> r.steps(ctx).stream()).distinct().toList();
    }
}
```

A.2 Aggregate with guarded state transitions and events

```
public class PurchaseOrder extends AggregateRoot<PoId> {
    private PoStatus status = PoStatus.DRAFT;
    private final List<PoLine> lines = new ArrayList<>();

    public void issue(Clock clock, UserId by) {
        require(status == PoStatus.APPROVED, PoErrors.invalidTransition(id, status, PoStatus.ORDERED));
        require(!lines.isEmpty(), PoErrors.noLines(id));
        this.status = PoStatus.ORDERED;
        this.issuedAt = clock.instant();
        registerEvent(new PurchaseOrderIssued(id, vendorId, total(), by, issuedAt));
    }

    public void registerReceipt(PoLineId line, Quantity qty) {
        PoLine l = lineById(line);
        l.receive(qty);                                 // throws if over-receipt beyond tolerance
        status = allReceived() ? PoStatus.RECEIVED : PoStatus.PARTIALLY_RECEIVED;
        registerEvent(new GoodsReceived(id, line, qty));
    }
}
```

A.3 Application service (use case) with transaction + outbox

```
@Service
class IssuePurchaseOrderHandler {
    private final PurchaseOrderRepository repo;
    private final DomainEventPublisher events;      // writes to outbox in same transaction
    private final Clock clock;

    @Transactional
    @PreAuthorize("hasAuthority('PO_ISSUE')")
    public PurchaseOrderSummary handle(IssuePoCommand cmd, CurrentUser user) {
        PurchaseOrder po = repo.findById(cmd.poId()).orElseThrow(() -> notFound(cmd.poId()));
        po.issue(clock, user.id());
        repo.save(po);                                  // optimistic lock via @Version
        events.publishAll(po.pullEvents());             // Outbox rows persisted atomically
        return PurchaseOrderSummary.from(po);
    }
}
```

A.4 Transactional outbox relay

```
@Component
class OutboxRelay {
    @Scheduled(fixedDelayString = "${outbox.poll-ms:500}")
    @SchedulerLock(name = "outbox-relay", lockAtMostFor = "PT30S")
    void publishPending() {
        List<OutboxEvent> batch = outboxRepo.lockNextBatch(100);   // FOR UPDATE SKIP LOCKED
        for (OutboxEvent e : batch) {
            try {
                broker.send(e.topic(), e.key(), e.payload());
                e.markPublished(clock.instant());
            } catch (Exception ex) {
                e.recordFailure(ex.getMessage());                  // retry with backoff
            }
        }
    }
}
```

A.5 Idempotent payment endpoint (sketch)

```
@PostMapping("/payments")
ResponseEntity<PaymentResponse> pay(@RequestHeader("Idempotency-Key") String key,
                                    @Valid @RequestBody PayRequest body) {
    return idempotency.execute(key, hash(body), () -> {
        Payment p = paymentService.pay(body.toCommand());
        return ResponseEntity.status(CREATED).body(PaymentResponse.from(p));
    });   // same key + same hash -> stored response; same key + different hash -> 422
}
```

A.6 ArchUnit rules

```
@AnalyzeClasses(packages = "com.procureflow")
class ArchitectureTest {
    @ArchTest static final ArchRule domain_is_pure =
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "org.springframework..", "jakarta.persistence..", "com.fasterxml..");

    @ArchTest static final ArchRule no_cycles_between_modules =
        slices().matching("com.procureflow.(*)..").should().beFreeOfCycles();

    @ArchTest static final ArchRule controllers_use_application_layer =
        noClasses().that().resideInAPackage("..web..")
            .should().dependOnClassesThat().resideInAPackage("..infrastructure..");
}
```

Appendix B - Glossary

Term	Meaning
P2P	Procure-to-Pay: the process from requesting a purchase to paying the vendor
Requisition	Internal request to buy goods or services, needs approval
RFQ	Request for Quotation sent to vendors to get prices
PO	Purchase Order: a binding order sent to a vendor
GRN	Goods Received Note: proof that goods arrived
Three-way match	Comparing PO, GRN and invoice before paying
Aggregate	DDD cluster of objects changed as one consistency unit with a single root
Outbox	Pattern storing events in the DB with the state change, then publishing them reliably
Idempotency	Repeating an operation has the same effect as doing it once
ADR	Architecture Decision Record: short note capturing a decision and its reasons

Appendix C - Recommended Reading

- Eric Evans - Domain-Driven Design; Vaughn Vernon - Implementing Domain-Driven Design (aggregates, events).
- Robert C. Martin - Clean Architecture (SOLID, boundaries); Alistair Cockburn - Hexagonal Architecture article.
- Chris Richardson - Microservices Patterns (outbox, saga, idempotent consumer).
- Martin Kleppmann - Designing Data-Intensive Applications (consistency, messaging).
- Spring Modulith documentation (module verification and event publication registry); an alternative to hand-written module rules.
- OWASP ASVS and the OWASP Top 10 for API security.